进芯 ADVCHIP-SPI 数据监测平台

用户说明文档

Rev. V 1.0，12/2025

应用方案部

湖南进芯电子科技有限公司

2025 年 12 月

技术文件修订记录

版本号	时间	描　述	制作人	审核人	批准人
V1.0	2025-12-15	首次发布	古小琴		

目录

一.上位机概述 4

1.1 软件概述 4

1.2 功能框架 4

1.3 运行环境 4

1.4 使用方法 5

二.下位机概述 9

2.1 下位机软件移植及注意事项 9

2.2 下位机待观测数据的调整方法 10

一.上位机概述

1.1 软件概述

- 本手册将主要介绍了 Advchip-SPI 数据监测平台的用户操作方法以及注意事项。
- 软件名称：Advchip-SPI 数据监测平台平台
- 软件介绍 ：通过 USBSPI 实现 PC 与下位机 DSP 通讯，高速采集下位机 DSP 发送的数据，最高可采集 20KHz（50μs 周期）数据流，软件界面对参数进行显示以及监测分析，同时数据在虚拟示波器下可实时曲线显示，数据缩放以及测量，且所采集数据可储存成 CSV 文件。

1.2 功能框架

Advchip-SPI 电机调试平台主要分为 4 个功能模块

通讯模块：软件采用 USPSPI 通讯，模块包含了设备打开、关闭、通讯超时检测以及通讯数据异常检测等。

状态显示：实时显示 DSP 上传的运行参数状态信息，共支持 10 个参数显示，参数名可通过文件 LIST.TXT 自定义。

实时数据：该页面数据为用户自定义的监测变量，可支持 3 个自定义变量，此页面数据更新周期最小可为 50us。

虚拟示波器：此页面可实时显示普通参数或者实时数据的曲线，支持 4 通道数据波形显示，用户可自定义坐标轴范围、对曲线进行缩放平移，测量。

数据保存：收到的数据都将保存成 CSV 文件格式，以保存时间命名。

1.3 运行环境

本软件可在 PC windows X86 以及 X64 平台下使用，推荐配置：双核 2 GHz 以上处理

器，4 GB 及以上内存。

1.4 使用方法

1.4.1 安装/运行

该软件为免安装版本，直接运行软件包中的 EXE 程序即可，软件包含如下图 1 个文件

名称	修改日期	类型	大小
Advchip-SPI数据监测平台V1.0	2025-12-12 19:25	应用程序	10,002 KB
libusb-1.0.dll	2022-03-21 20:01	应用程序扩展	134 KB
USB2XXX.dll	2022-10-26 10:22	应用程序扩展	320 KB
list	2025-12-12 18:47	文本文档	1 KB

图 1

1.4.2 运行软件

1、软件页面如下图 2 所示：

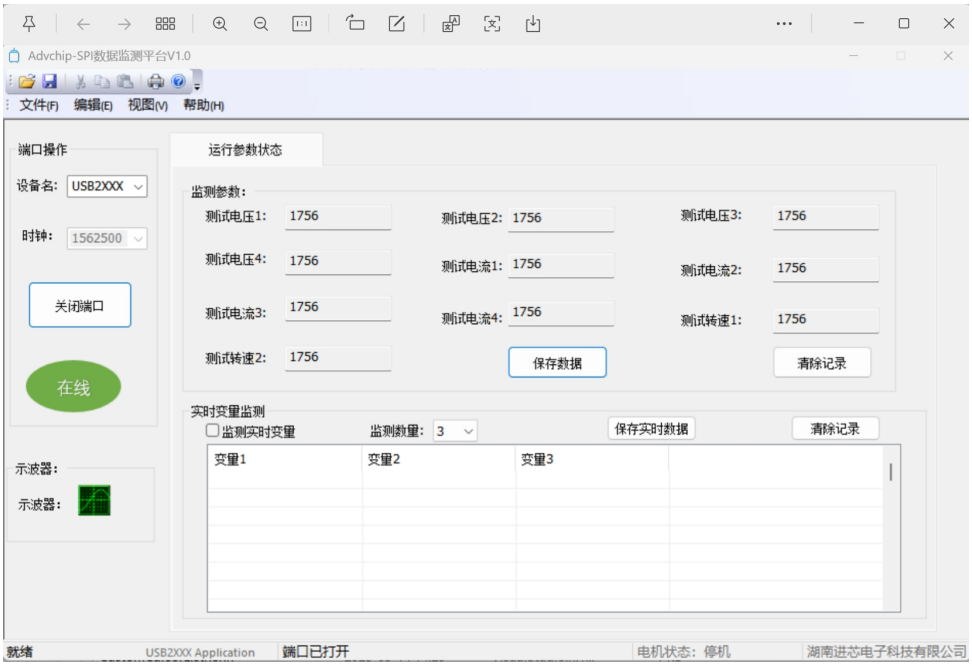


图 2

10 个参数名称可通过程序文件下下的 LIST.TXT 进行自定义设置，按顺序从参数 1-参数 10 一次设置保存后再重新打开软件即可生效，如下图 3 所示：

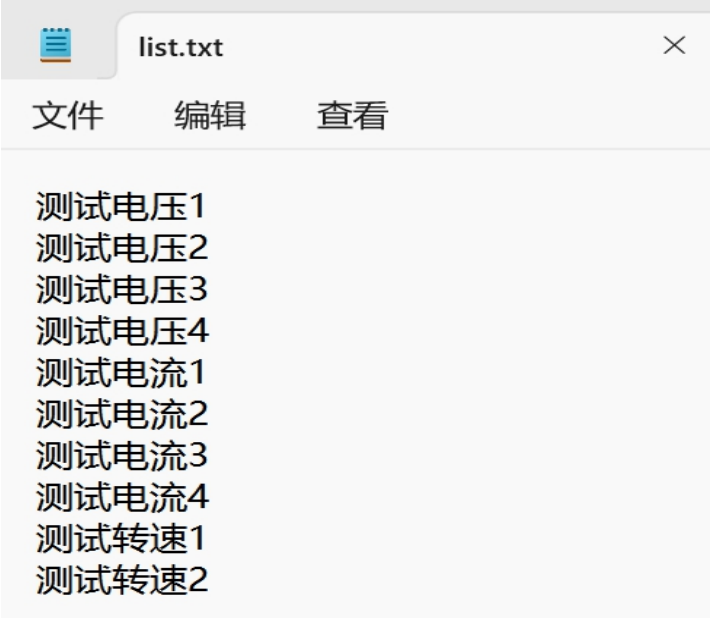


图 3

2、连接好硬件后直接打开端口，成功后即左下角显示在线状态，并读取下位机参数至页面显示，如下图 4 所示，点击保存数据可将数据存成 CSV 格式，文件名以保存时间命令，如“20251119-14-11-46.CSV”，文件在程序所在文件目录下。

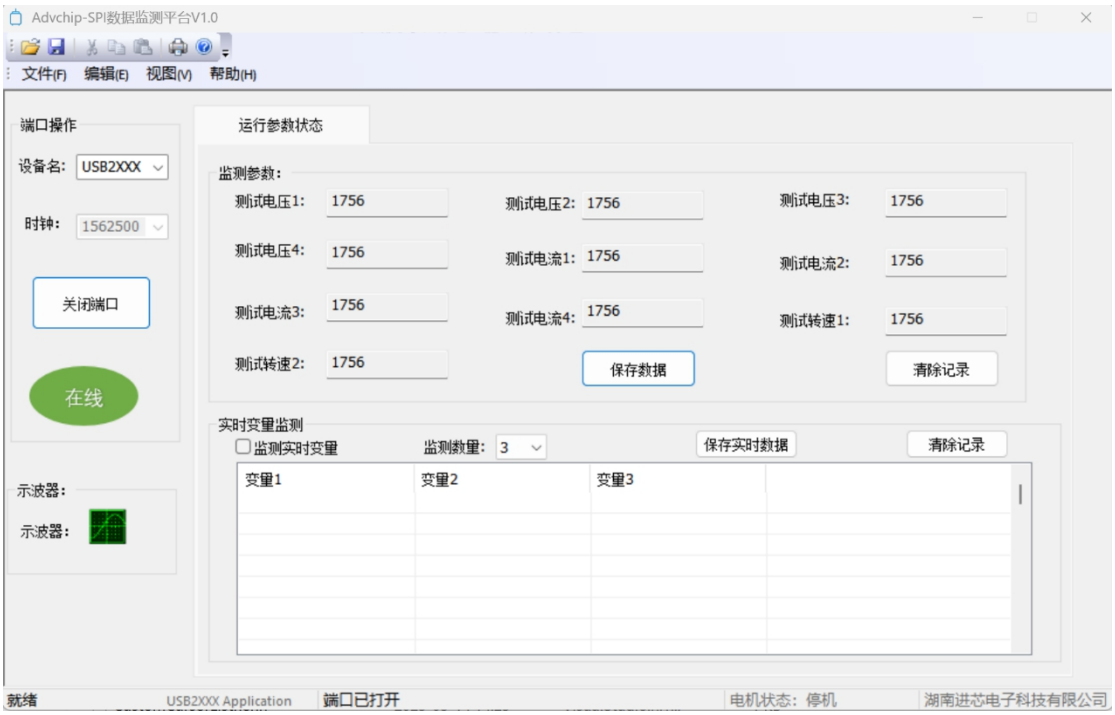


图 4

3、实时变量监测:

监测进行中的显示页面如下图 5 所示：

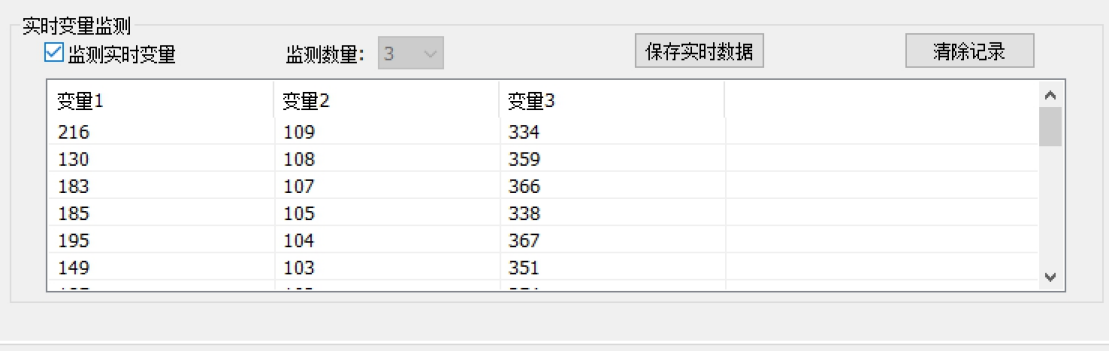


图 5

点击保存实时数据按钮可保存当前实时数据，保存路径为应用程序所在文件夹下，文件名为保存时间加上后缀_Rt，如“20251119-14-31-46_Rt.CSV”。

注意：该页面下的普通状态数据传输为 10ms 周期更新，实时变量监测时数据传输最快可达 50us，选择实时变量监测后将停止普通状态数据上传更新。

4、虚拟示波器页面

点击软件的第一个页面中的示波器对应图标按钮即可打开示波器页面，如下图 6 图 7 所示：

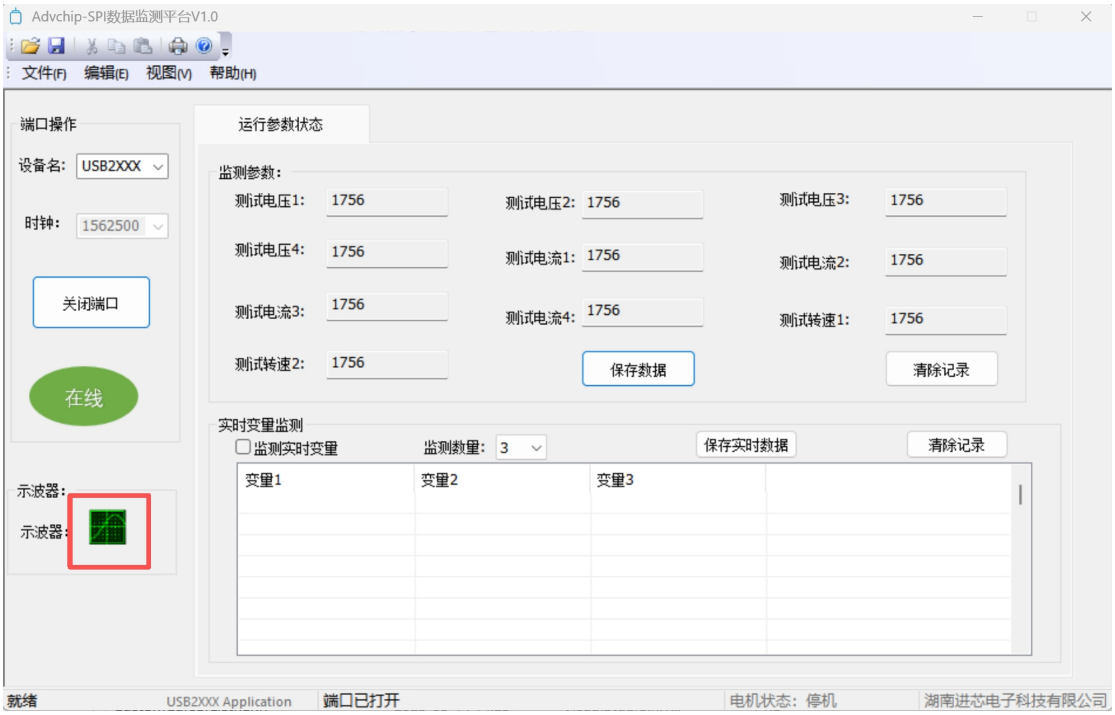


图 6

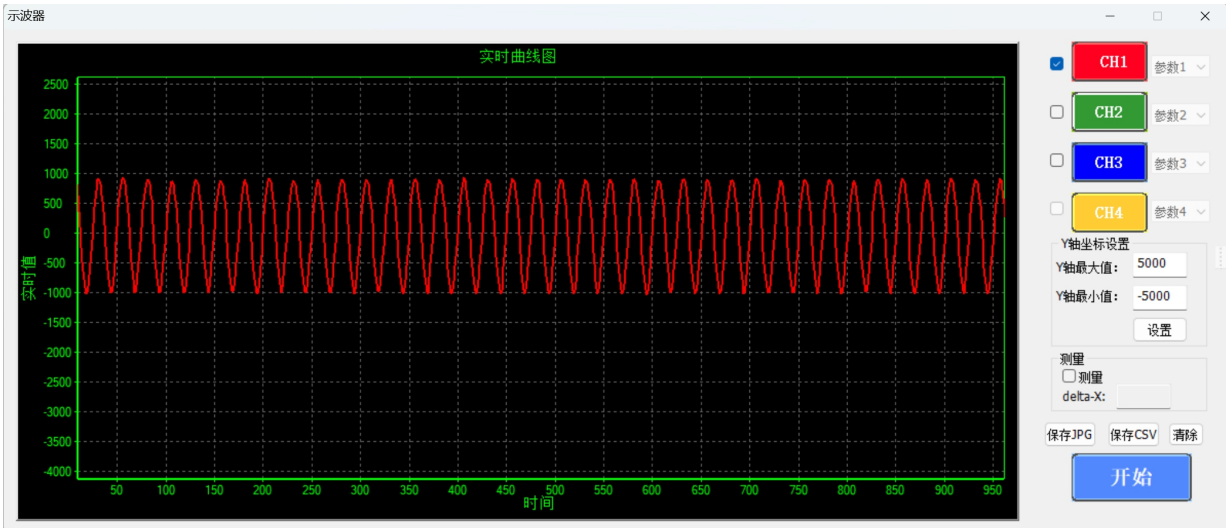


图 7

示波器一共有 4 个通道可供用户选择观测，在普通状态数据传输时（10ms 每帧数据），每个通道可选择状态页面上显示的参数 1-参数 10，Y 轴坐标用户可自行设置，输入好数值后点击设置或者回车均可确认设置。 用户可对曲线进行平移缩放测量等操作，同时对波形进行图片保存或者 CSV 格式数据保存。

当用户选择了实时变量监测功能时 ,默认通道 1-通道 3 对应变量 1-变量 3 的曲线显示，如下图 8 所示：

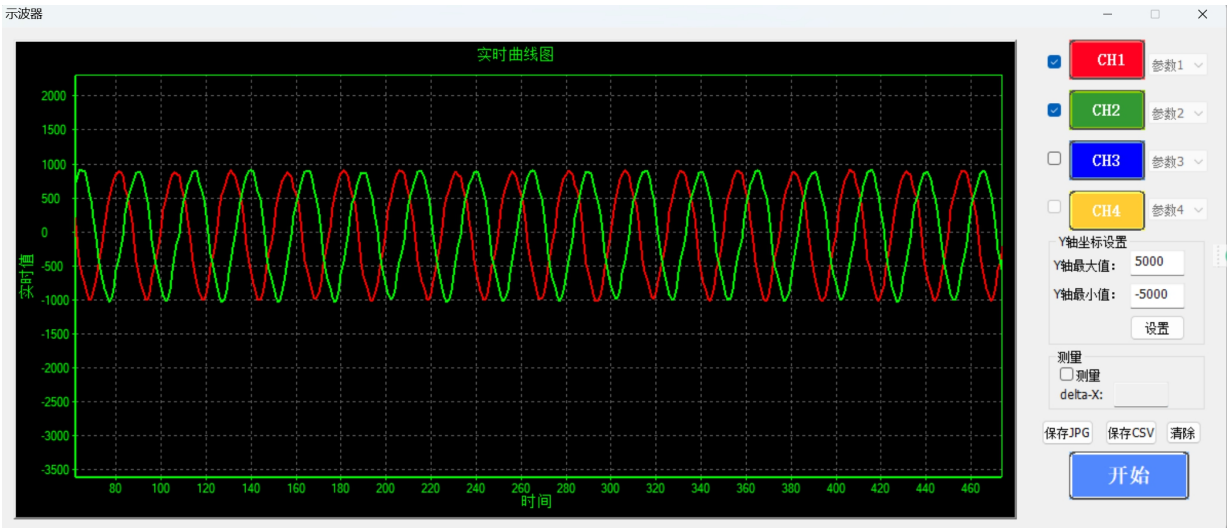


图 8

二.下位机概述

该版本下位机可适用与多款具有 SPI 通信功能的芯片，采用芯片端做主机，PC 端做从机的方式，波特率配置 1MHz。

2.1 下位机软件移植及注意事项

该版本下位机程序都集中存放在 CommInterface_SPI.c 中，内容包括 SPI 初始化、SPI 中断服务函数、10 个变量数据传输函数（20ms 发送一次数据）、3 个变量数据传输函数（PWM 或 ADC 中断中执行）。

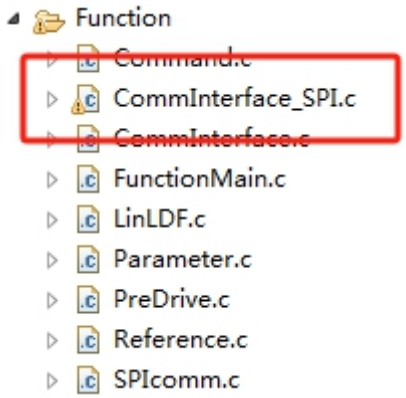


图 9

移植时仅需要将 CommInterface_SPI.c 和 CommInterface_SPI.h 两个文件移植进新工程，
在系统模块初始化中调用 Init_Spi 函数，
在主循环中调用 CommInteraction 函数，
在 PWM 或 ADC 中断中调用 SendRTFrame 函数。

```
void InitPeripherals(void)
{
    InitSetGpio();
//    InitSpiIO();
//      InitI2CIO();
    InitSetPWM();
    InitSetAdc();
    //InitSpi();
    Init_Spi();
    //InitI2C();
    PreDriverInit();
    DriverClaerFault();
    // Set up SCI function...
```

图 10

```
        if((Level500usCount & 0x03) == 2)
        {
            gDSPActiveTime.Function2MsBase = GetTime();
            SystemLeve2msFunction();
            gDSPActiveTime.Function2Ms = gDSPActiveTime.Function2MsBase - GetTime();
            CommInteraction();
        }
    }
```

图 11

```
interrupt void ZeroOfEPWMISR(void)
{
    EPwm2Regs.ETCLR.bit.INT = 1;

    EINT;

    ADCOverInterrupt();

    SendRTFrame();

    DINT;
```

图 12

2.2 下位机待观测数据的调整方法

```
/*************************************************************/
/*                  观测变量填充函数（慢速）                 */
/*************************************************************/
//低速观测变量 共10个 [2]~[11]
void FillSysStateData(void)
{
    unsigned int CRC_SysStateData;
    SysStateData[0] = 0x55AA;
    SysStateData[1] = 0x0003;
    SysStateData[2] = Test_MsCnt;          //DATA1
    SysStateData[3] = Test_MsCnt;          //DATA2
    SysStateData[4] = Test_MsCnt;          //DATA3
    SysStateData[5] = Test_MsCnt;          //DATA4
    SysStateData[6] = Test_MsCnt;          //DATA5
    SysStateData[7] = Test_MsCnt;          //DATA6
    SysStateData[8] = Test_MsCnt;          //DATA7
    SysStateData[9] = Test_MsCnt;          //DATA8
    SysStateData[10] = Test_MsCnt;         //DATA9
    SysStateData[11] = Test_MsCnt;         //DATA10
    CRC_SysStateData = CRC_16(SysStateData,12);
    SysStateData[12] = CRC_SysStateData;
    Test_MsCnt++;                          //自测试变量
}
```

图 13

FillSysStateData 函数中，数组 SysStateData[2]~SysStateData[11]对应 10 个待观测数据。

```
void SendRTFrame(void)                    //使用时需要放入到adc中断或PWM中断服务函数中
{
    Uint16 sum = 0;
    Uint16 rxdata = 0;

    if(EnRTVar == 1)
    {
        SpiaRegs.SPITXBUF = 0x55aa;
        while(SpiaRegs.SPISTS.bit.INT_FLAG == 0);
        rxdata = SpiaRegs.SPIRXBUF;

        SpiaRegs.SPITXBUF = 0x2;
        while(SpiaRegs.SPISTS.bit.INT_FLAG == 0);
        rxdata = SpiaRegs.SPIRXBUF;

        SpiaRegs.SPITXBUF = gCurSamp.U;              //该位为实时变量1  DATA1
        while(SpiaRegs.SPISTS.bit.INT_FLAG == 0);
        rxdata = SpiaRegs.SPIRXBUF;

        SpiaRegs.SPITXBUF = gCurSamp.V;              //该位为实时变量2  DATA2
        while(SpiaRegs.SPISTS.bit.INT_FLAG == 0);
        rxdata = SpiaRegs.SPIRXBUF;

        SpiaRegs.SPITXBUF = gCurSamp.W;              //该位为实时变量2  DATA2
        while(SpiaRegs.SPISTS.bit.INT_FLAG == 0);
        rxdata = SpiaRegs.SPIRXBUF;

        sum = gCurSamp.U + gCurSamp.V + gCurSamp.W + 0x55aa + 0x2;    //该位为校验位 sum = DATA1 + DATA2 + DATA3 + 0x55aa + 0x2;

        SpiaRegs.SPITXBUF = sum;
        while(SpiaRegs.SPISTS.bit.INT_FLAG == 0);
        rxdata = SpiaRegs.SPIRXBUF;
```

图 14

SendRTFrame 函数用于传输 3 个对实时性要求较高的数据，填充寄调整方法需依照注释填写。